The Journal of Open Source Software

fluidfoam: from OpenFOAM cases to NumPy arrays in one call

Cyrille Bonamy [1], **Pierre Augier** [1], **Quentin Clemencot**[3], **Eduard Puig Montellà** [4], **Rémi Chassagne** [1], **and Julien Chauchat** [2]

1 Université Grenoble Alpes, CNRS, LEGI, Grenoble, France **2** Université Grenoble Alpes, INRAE, IRD, CNRS, IGE, Grenoble, France **3** Alstom, France **4** Centro Oceanográfico de Canarias, Instituto Español de Oceanografía, Consejo Superior de Investigaciones Científicas (IEO-CSIC), Santa Cruz de Tenerife, Spain

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 07 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

In the computational fluid dynamics (CFD) domain, OpenFOAM (OpenCFD Ltd., n.d.; OpenFOAM Foundation, n.d.) has become one of the most popular open-source toolboxes for researchers, engineers, and students. It provides solvers for a very wide range of applications - aerodynamics, heat transfer, combustion, chemical engineering, and environmental flows alike. Each simulation produces large output files in OpenFOAM's own file format, which must then be analysed to compute statistics, plot profiles, or compare with experiments.

fluidfoam is a Python package that reads these output files directly, converting meshes and flow fields into NumPy arrays (Harris et al., 2020), the standard array format of the scientific Python ecosystem, in a single function call. It requires neither an OpenFOAM installation nor conversion to intermediate formats, and runs on a laptop analysing data copied from a supercomputer. Results from parallel simulations, which are split across multiple directories, are reassembled transparently. Beyond reading, fluidfoam processes the outputs of OpenFOAM function objects (forces, probes) and writes and reads 1D profiles for boundary-condition imposition via swak4Foam (Gschaider, n.d.). The package is pure Python and released under GPL-3.0. Its core depends only on NumPy and SciPy (Virtanen et al., 2020); Matplotlib (Hunter, 2007) is an optional dependency, used only for plotting.

Statement of need

Post-processing is a significant fraction of the working time of CFD researchers. The native OpenFOAM toolchain is oriented towards interactive visualisation in ParaView (Ayachit, 2015) or requires converting data to intermediate formats (VTK, HDF5) before analysis. Many practitioners instead analyse results in Python, where the scientific ecosystem makes it easy to compute statistics and spectra, to apply modal decompositions to snapshot matrices, and to couple simulation output with data-driven workflows.

fluidfoam is designed for researchers and students who need scriptable, reproducible post-processing of OpenFOAM results: research groups running parametric studies, teachers preparing course material, and analysts on machines where OpenFOAM itself is not installed. Studies of canonical flows typically read tens to hundreds of time steps of several fields, often from decomposed parallel runs stored on network file systems, so reading files is a large share of the analysis time; the package aims to keep this cost modest and to hide the bookkeeping of parallel layouts from the user.

DRAFT

State of the field

Several Python tools address OpenFOAM data, with different emphases. PyFoam (Gschaider, 2023) focuses on case configuration and running rather than analytical post-processing. Ofpp (Xianghua, 2023) is a minimal reader that appears to be no longer actively maintained (no commit since August 2023). fluidsimfoam (Augier & Danaeifar, 2025) sets up and runs simulations within the fluidsim framework, relying on HDF5 output rather than native OpenFOAM files. foamlib (Gerlero & Kler, 2025) is a comprehensive, fully type-hinted library for interacting with OpenFOAM files and running cases, including asynchronous and Slurm-based execution. Turbulucid (Mukha, 2019) provides scripted post-processing and plotting of plane 2D simulations based on VTK. Generic readers such as the OpenFOAM reader of ParaView, accessible from Python through PyVista (Sullivan & Kaszynski, 2019), give access to cases through the VTK data model. A different approach is taken by PythonFOAM (Maulik et al., 2021) and by embedded-interpreter approaches (Rodriguez & Cardiff, 2022), which call Python from within the solver for in-situ analysis; these require compiling against OpenFOAM and are complementary to the offline post-processing addressed here.

To our knowledge, fluidfoam offers a specific combination of features that is not found together in these tools: (i) reconstruction of decomposed parallel cases directly into global, ordered fields; (ii) automatic reshaping of structured meshes into multi-dimensional arrays; and (iii) a minimal dependency footprint (the core needs only NumPy and SciPy; Matplotlib is optional and used only for plotting), which matters for shared analysis scripts, teaching, and continuous integration. These tools are largely complementary: a workflow may use foamlib or fluidsimfoam to launch simulations and fluidfoam to analyse their output.

Software design

The package is organised in layers. The reading functions (fields, meshes, patches) form the low-level core; on top of them, an OpenFoamSimu class loads, in one step, all fields saved at a given time step of a simulation (or of any simulation found in a directory tree), which is the typical entry point for exploratory analysis; and specialised modules handle postProcessing function-object output, mesh design and visualisation for simple configurations - including the generation of vector figures (SVG, PDF) of 2D meshes via Matplotlib (Hunter, 2007) line collections (the optional dependency), for publication-quality mesh illustrations. A dedicated module also closes the loop between one-dimensional and multi-dimensional simulations: it extracts vertical profiles of the flow fields from a 1D preliminary run and writes them in a format that swak4Foam (Gschaider, n.d.) can interpolate to impose boundary or initial conditions on a 2D/3D case. This 1D-to-3D workflow, classically used to initialise SedFoam studies (see the SedFoam tutorials (SedFoam developers, n.d.)), turns a costly 3D spin-up into a cheap 1D computation.

Three further design choices structure the core.

First, it is a standalone reader: it does not wrap OpenFOAM commands and does not link to the OpenFOAM C++ API. It reads the file format shared by the two OpenFOAM distributions (the OpenFOAM Foundation and OpenCFD), which removes any dependence on a particular version or distribution, at the cost of maintaining a parser for the OpenFOAM file format - a cost kept low because the format is stable and because the parser handles ASCII, binary (either byte order, taken from the FoamFile header), and gzip-compressed files uniformly.

A related, deliberate goal was to keep the code small and readable. The reading core is a single module of about 1,800 lines of plain Python, and the whole package fits in roughly 3,000 lines without metaprogramming, type machinery, or asynchronous constructs. Graduate students - a large share of the user base - can read the parser in an afternoon, adapt it to their particular output layout, or extend it; we consider this hackability a feature for research code, at the cost of the ergonomics that larger frameworks provide.

DRAFT

Second, decomposed parallel cases (uncollated and collated layouts) are reconstructed transparently using the cell, face, and boundary addressing files that OpenFOAM writes at decomposition time, so the same call returns identical arrays for reconstructed and decomposed data, and this reconstruction is shared by all functionalities. Reading of decomposed cases can optionally be multi-threaded, with bit-exact results. Since a silent reconstruction error would be invisible in downstream results, the transparent reconstruction is validated in the test suite against reconstructed references.

Third, a benchmark released with the package compares reading times with foamlib and with OpenFOAM's reconstructPar on fields of 10^6 and 8×10^6 cells (cold client cache, medians of 10 runs on dedicated compute nodes with an NFS file system, using fluidfoam 0.4.0, foamlib 1.10.0 and OpenFOAM v2606; scripts and raw outputs are archived with the release; timings vary by tens of percent between invocations, so ranges are given). fluidfoam is not the fastest reader of a single file: 26 ms against 14 ms for foamlib. For 10^6 cells decomposed into 64 non-contiguous subdomains, fluidfoam returns the global array in a single call in 0.15 to 0.21 s (0.06 s with 20 threads, an optional setting), against 0.29 to 0.48 s for foamlib, which does not reconstruct decomposed cases and needs about 30 lines of user code to reorder the cells with the addressing files. For 8×10^6 cells, single-threaded fluidfoam (0.73 s) and foamlib (0.54 to 0.83 s) are comparable, and eight threads bring fluidfoam to 0.38 s. Alternatively, reconstructPar can be run once for the field (2.5 s and 9.6 s for the two cases), after which the reconstructed field is read in 0.04 s and 0.32 s: this pays off only if the same field is read more than 14 to 24 times, and requires an OpenFOAM installation and a second copy of the data on disk. All reported results match a ground-truth array. The benchmark also yields practical guidance: write results in binary format and load each time series once.

Research impact statement

fluidfoam has been developed since 2017 by the SedFoam development team as a Python API to interact with OpenFOAM simulations, initially as the post-processing layer of SedFoam (Chauchat et al., 2017), a two-phase sediment-transport model used in morphodynamics research. It has since been used to analyse simulations in studies of unsteady effects in oscillatory sheet flows (Mathieu et al., 2022), finite-size effects in two-fluid large-eddy simulation of particle-laden boundary layers (Mathieu et al., 2021), a frictional–collisional model for bedload transport (Chassagne et al., 2023), and submerged granular column collapse (Puig Montellà et al., 2023). More recently, it has also been used for the post-processing of OpenFOAM simulations of swash dynamics over rough coastal surfaces (Kaczmarek et al., 2026). It is also the post-processing layer of the group's other solvers and methods: sedInterFoam, a three-phase model for free-surface sediment transport (Mathieu et al., 2025); sedExnerFoam, an Exner-based morphodynamics model (Renaud et al., 2026); and an Exner diagnosis method for two-fluid morphodynamics simulations (Gilletta et al., 2026). In the SedFoam tutorials, it is the interface for reading OpenFOAM output (SedFoam developers, n.d.).

Beyond the group, the package is distributed on PyPI and conda-forge. As of 22 October 2026, it has been downloaded over 125,000 times from PyPI since its first release (about 800 times per month, excluding mirrors) and 376 times from conda-forge over the last six months. Its GitHub repository has 201 stars and 54 forks, and a Google Scholar full-text search finds it mentioned in 36 publications, after removing homonyms.

fluidfoam is also used beyond the SedFoam solvers, in tidal energy, machine learning and teaching. In a monopile scour study under tidal flow, it reads the velocity and pressure fields at the end of each half tidal cycle to initialise the next one (Liu et al., 2024), and a machine-learning study of transient flow prediction around airfoils uses it to extract the grid and the flow-field data from the simulations (Xie et al., 2024). The independent package openfoam-postprocessing builds on fluidfoam for field access (Martins, 2025). A series of tutorials by another author presents proper orthogonal decomposition of OpenFOAM snapshots (Goswami, 2024c, 2025), function-object post-processing (Goswami, 2024a) and the extraction

DRAFT

and analysis of OpenFOAM data (Goswami, 2024b). fluidfoam is also listed in a survey of open-source CFD tools (Kodman et al., 2024). In a different application domain, it post-processes the simulations of 120 randomised blade shapes in an automated design methodology for cross-flow tidal turbines, developed in a PhD thesis supervised by one of the authors (Ruiz-Hussmann et al., 2025). The Sphinx-Gallery example gallery and the benchmark datasets provide reproducible materials for new users.

AI usage disclosure

In accordance with the JOSS AI usage policy, we disclose the following use of generative AI. Two assistants were used: the Mistral AI "Vibe Code" agent (model zai-glm-5-3) and Anthropic's Claude (model Claude Sonnet 5.5).

- *Paper text:* drafting and copy-editing of sections of this manuscript from source material provided by the authors.
- *Benchmark code:* generation of the comparative benchmark scripts and of synthetic OpenFOAM-format test datasets, with iterative debugging against real OpenFOAM output files.
- *Software:* the most recent changes to the fluidfoam codebase - essentially the transparent support of decomposed cases, the optional multi-threaded reading and a faster parsing of binary files - were made with AI assistance.
- *Documentation:* restructuring of the benchmark documentation.

AI was not used for interactions with editors or reviewers. The human authors reviewed and validated all AI-assisted outputs, ran the benchmarks on their own hardware, verified the reported numbers, and made all design decisions. All modified code is covered by the package's test suite and continuous integration. The authors remain fully responsible for the accuracy, originality, licensing, and ethical and legal compliance of all submitted materials.

Acknowledgements

The OpenFOAM simulations whose post-processing motivated and shaped fluidfoam were performed using HPC resources provided by GENCI (Grand Équipement National de Calcul Intensif) under projects A0160107567, A0172A14636, A0180107567, A0192A14636 and A0212A14636. We are thankful to the developers involved in OpenFOAM, and to all the students and colleagues at LEGI who used fluidfoam and contributed feedback and bug reports.

References

Augier, P., & Danaeifar, P. (2025). *Fluidsimfoam: Python framework for OpenFOAM*. https://foss.heptapod.net/fluiddyn/fluidsimfoam

Ayachit, U. (2015). *The ParaView guide: A parallel visualization application*. Kitware. ISBN: 978-1930934306

Chassagne, R., Bonamy, C., & Chauchat, J. (2023). A frictional–collisional model for bedload transport based on kinetic theory of granular flows: Discrete and continuum approaches. *Journal of Fluid Mechanics*, *964*, A27. https://doi.org/10.1017/jfm.2023.335

Chauchat, J., Cheng, Z., Nagel, T., Bonamy, C., & Hsu, T.-J. (2017). SedFoam-2.0: A 3-D two-phase flow numerical model for sediment transport. *Geoscientific Model Development*, *10*(12), 4367–4392. https://doi.org/10.5194/gmd-10-4367-2017

Gerlero, G. S., & Kler, P. A. (2025). Foamlib: A modern Python package for working with OpenFOAM. *Journal of Open Source Software*, *10*(109), 7633. https://doi.org/10.21105/joss.07633

DRAFT

Gilletta, A., Bonamy, C., & Chauchat, J. (2026). Exner diagnosis method for two-fluid morphodynamics simulations. *International Journal of Multiphase Flow*, *194*, 105457. https://doi.org/10.1016/j.ijmultiphaseflow.2025.105457

Goswami, S. (2024a). *Conquer OpenFOAM post-processing with the power of function-objects and Python!* Blog post, 4 March 2024. https://goswami-13.github.io/posts/2024/02/blog-post-8/

Goswami, S. (2024b). *Dive deeper with fluidfoam: Advanced techniques for extracting & analyzing OpenFOAM data in Python*. Blog post, 11 March 2024. https://goswami-13.github.io/posts/2024/03/blog-post-9/

Goswami, S. (2024c). *Exploring proper orthogonal decomposition (POD) with OpenFOAM simulation data*. Blog post. https://goswami-13.github.io/posts/2024/04/blog-post-24/

Goswami, S. (2025). *3D POD and visualization with OpenFOAM and Python*. Blog post, 10 February 2025. https://goswami-13.github.io/posts/2025/02/blog-post-46/

Gschaider, B. (n.d.). *swak4Foam: General purpose utility for OpenFOAM*. https://openfoamwiki.net/index.php/Contrib/swak4Foam

Gschaider, B. (2023). *PyFoam: Python utilities for OpenFOAM*. https://pypi.org/project/PyFoam/

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kaczmarek, M., Michallet, H., Bonamy, C., & Chauchat, J. (2026). Numerical study of swash dynamics: Development of a porous layer boundary treatment for rough surfaces modelling. *Coastal Engineering*, *208*, 105004. https://doi.org/10.1016/j.coastaleng.2026.105004

Kodman, J. B., Singh, B., & Murugaiah, M. (2024). A comprehensive survey of open-source tools for computational fluid dynamics analyses. *Journal of Advanced Research in Fluid Mechanics and Thermal Sciences*, *119*(2), 123–148. https://doi.org/10.37934/arfmts.119.2.123148

Liu, J., Lu, J., & Liang, Z. (2024). Computational simulation of monopile scour under tidal flow considering suspended energy dissipation. *Water*, *16*(14), 1940. https://doi.org/10.3390/w16141940

Martins, F. (2025). *Openfoam-postprocessing: OpenFOAM field access, local interpolation and structured-array post-processing in Python*. GitHub repository. https://github.com/EngFlavioMartins/openfoam-postprocessing

Mathieu, A., Chauchat, J., & Bonamy, C. (2022). Numerical investigation of unsteady effects in oscillatory sheet flows. *Journal of Fluid Mechanics*, *942*, A25. https://doi.org/10.1017/jfm.2022.405

Mathieu, A., Chauchat, J., Bonamy, C., Balarac, G., & Hsu, T.-J. (2021). A finite-size correction model for two-fluid large-eddy simulation of particle-laden boundary layer flow. *Journal of Fluid Mechanics*, *913*, A26. https://doi.org/10.1017/jfm.2021.4

Mathieu, A., Kim, Y., Hsu, T.-J., Bonamy, C., & Chauchat, J. (2025). sedInterFoam 1.0: A three-phase numerical model for sediment transport applications with free surfaces. *Geoscientific Model Development*, *18*(5), 1561–1573. https://doi.org/10.5194/gmd-18-1561-2025

Maulik, R., Fytanidis, D. K., Lusch, B., Vishwanath, V., & Patel, S. (2021). *PythonFOAM: In-situ data analyses with OpenFOAM and Python*. https://arxiv.org/abs/2103.09389

DRAFT

Mukha, T. (2019). Turbulucid: A Python package for post-processing of fluid flow simulations. *Journal of Open Research Software*, *7*(1). https://doi.org/10.5334/jors.213

OpenCFD Ltd. (n.d.). *OpenFOAM | the open source CFD toolbox*. https://www.openfoam.com/

OpenFOAM Foundation. (n.d.). *OpenFOAM: The open source CFD toolbox*. https://openfoam.org/

Puig Montellà, E., Chauchat, J., Bonamy, C., Weij, D., Keetels, G. H., & Hsu, T. J. (2023). Numerical investigation of mode failures in submerged granular columns. *Flow*, *3*, E28. https://doi.org/10.1017/flo.2023.23

Renaud, M., Bonamy, C., Bertrand, O., & Chauchat, J. (2026). sedExnerFoam 2412: A 3D Exner-based sediment transport and morphodynamics model. *Geoscientific Model Development*, *19*(6), 2299–2331. https://doi.org/10.5194/gmd-19-2299-2026

Rodriguez, S., & Cardiff, P. (2022). A general approach for running Python codes in OpenFOAM using an embedded pybind11 Python interpreter. *OpenFOAM Journal*, *2*, 166–182. https://doi.org/10.51560/ofj.v2.79

Ruiz-Hussmann, K., Delafin, P.-L., Bennecke, T., Bonamy, C., & Hoerner, S. (2025). Blade shape evaluation for cross-flow tidal turbines: A fast and automated assessment tool for early design stage based on power and load coefficients. *Physics of Fluids*, *37*(11), 117111. https://doi.org/10.1063/5.0291030

SedFoam developers. (n.d.). *SedFoam: Documentation and tutorials*. Online documentation. https://sedfoam.github.io/

Sullivan, C. B., & Kaszynski, A. A. (2019). PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). *Journal of Open Source Software*, *4*(37), 1450. https://doi.org/10.21105/joss.01450

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Xianghua, X. (2023). *Ofpp: OpenFOAM Python parser*. https://github.com/xu-xianghua/ofpp

Xie, H.-R., Wang, Z.-Q., Li, Y.-B., Li, Q., Wu, W.-T., Han, J.-L., Peng, J.-Z., & He, Y. (2024). Fast spatiotemporal sequence graph convolutional network-based transient flow prediction around different airfoils. *Physics of Fluids*, *36*(10), 105114. https://doi.org/10.1063/5.0226082

DRAFT